



**Università
di Brescia**

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Corso di Laurea Magistrale

in Ingegneria Informatica

Relazione di Progetto del Corso

Algoritmi e Strutture Dati

Docente: Prof.ssa Marina Zanella

Studenti:

Davide Leone - 723335

Filippo Camossi - 734170

Anno Accademico 2024/2025

Indice

1	Introduzione	1
1.1	Strumenti e scelte tecnologiche	2
1.2	Rappresentazione dello stato della cella	2
2	Compito uno: generazione dell'ambiente	4
2.1	Personalizzazione della griglia tramite parametri	4
2.2	Tipologie di ostacoli e modellizzazione logica	4
2.3	Algoritmo di generazione	5
3	Compito due: raggiungibilità e distanza libera	8
3.1	Distanza libera (d_{lib}) e formulazione matematica	8
3.2	Contesto e complemento	9
3.3	Proiezione di raggi geometrici ($O(\max(R, C)^2)$)	10
3.4	Frontiera della chiusura	12
4	Compito tre: ricerca del cammino minimo	14
4.1	Definizione ricorsiva e flusso esecutivo	14
4.2	Analisi della potatura: riga 16 contro riga 17	14
4.3	Potatura branch-and-bound globale	15
4.4	Controllo preliminare di raggiungibilità con i componenti connessi . .	17
4.5	Ottimizzazioni fisiche e implementative	18
4.6	Requisiti funzionali implementati	19
5	Compito quattro: sperimentazione e analisi	22
5.1	Analisi di complessità spaziale asintotica	22
5.2	Analisi di complessità temporale asintotica	22
5.3	Nota metodologica sulla misura	23
5.4	Analisi dei risultati sperimentali e discussione dei grafici	23
5.4.1	1. Tempo in funzione della densità di ostacoli	24
5.4.2	2. Confronto delle configurazioni (potatura e ordinamento) . .	25

5.4.3	3. Crescita temporale asintotica (scala bilogaritmica)	27
5.4.4	4. Correlazione tra tempo e celle di frontiera	29
5.4.5	5. Correlazione tra tempo e invocazioni ricorsive	30
5.4.6	6. Verifica formale della simmetria	31
5.5	Oracolo di correttezza indipendente: confronto con A^*	32
5.6	Divario di ottimalità dei risultati interrotti (anytime)	33
5.7	Confronto di strutture dati: matrice di byte contro piani di bit	34
6	Conclusioni	36
6.1	Limitazioni riscontrate	36
6.2	Motivazioni delle scelte implementative	37
6.3	Sviluppi futuri	38
7	Uso del codice	39
7.1	Configurazione e requisiti di sistema	39
7.2	Configurazione del file JSON delle griglie	39
7.3	Guida all'uso della riga di comando	40
7.3.1	1. Generazione di griglie (Compito 1)	40
7.3.2	2. Calcolo di contesto, complemento e frontiera (Compito 2) .	40
7.3.3	3. Calcolo della distanza ideale d_{lib}	41
7.3.4	4. Calcolo del cammino minimo (Compito 3)	41
7.3.5	5. Campagna sperimentale completa (Compito 4)	42
7.3.6	6. Verifica di simmetria formale	42
7.4	Rigenerazione delle figure pedagogiche della relazione	42
7.5	Esecuzione della batteria di prove unitarie	43

1 Introduzione

La presente relazione descrive la progettazione, l'implementazione e la campagna sperimentale condotta per il sistema di navigazione e calcolo del cammino minimo su griglia bidimensionale, sviluppato in conformità con i requisiti del corso di Algoritmi e Strutture Dati per l'anno accademico 2024/25.

L'elaborato affronta il calcolo del cammino minimo su griglie bidimensionali in presenza di ostacoli. Il sistema mira a limitare il sovraccarico computazionale tipico delle esplorazioni ricorsive ricorrendo a strutture dati specifiche e a regole di potatura spaziale.

La figura 1 anticipa in quattro pannelli l'intero percorso seguito dalla relazione, sulla stessa istanza di griglia: si parte da uno spazio libero senza ostacoli, si generano proceduralmente gli ostacoli (Compito 1), si calcola dall'origine O la chiusura raggiungibile con cammini liberi (in verde/arancio) individuandone la frontiera (cerchiata) tramite la proiezione dei raggi cardinali e diagonali (Compito 2), e infine si risolve il cammino minimo completo verso la destinazione D componendo più cammini liberi lungo la sequenza dei landmark (Compito 3). I capitoli successivi riprendono singolarmente ciascuno di questi passaggi, concludendo con la valutazione delle prestazioni (Compito 4) e con le considerazioni finali.

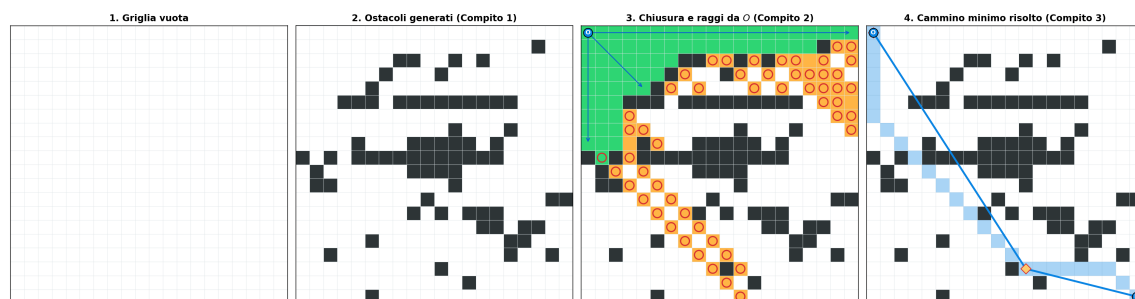


Figura 1: Panoramica del sistema sulla stessa griglia: dallo spazio vuoto al cammino minimo risolto.

1.1 Strumenti e scelte tecnologiche

Il software è scritto in **Python 3.13** e utilizza:

- **NumPy**: gestisce la rappresentazione della griglia attraverso array multidimensionali, sfruttando le operazioni vettoriali per ottimizzare le letture e le modifiche dello stato;
- **Matplotlib**: genera le visualizzazioni grafiche dei percorsi e produce i grafici per l'analisi delle prestazioni (Compito 4);
- **Argparse**: fornisce l'interfaccia a riga di comando.

Rispetto all'approccio a oggetti, che associa a ogni cella un'istanza dedicata e grava sul gestore della memoria, la rappresentazione tramite array numerici lavora su dati contigui in memoria.

1.2 Rappresentazione dello stato della cella

Ogni cella della griglia è codificata con un numero intero all'interno di una matrice NumPy bidimensionale di tipo `uint8` (1 byte per cella) [1], in base allo schema seguente:

Tabella 1: Rappresentazione dello stato della cella a livello logico e fisico

Stato cella	Valore fisico	Significato operativo
Libera	0	Cella attraversabile (navigabile).
Ostacolo fisso	1	Ostacolo permanente generato nel Compito 1 o importato da JSON.
Ostacolo temporaneo	2	Cella della chiusura corrente marcata direttamente in memoria durante il ritracciamento ricorsivo; ripristinata a 0 in fase di risalita.

Questa codifica numerica consente il **ritracciamento (backtracking) sul posto**. A ogni chiamata ricorsiva di `CAMMINOMIN`, invece di clonare la matrice del-

la griglia (operazione che esaurirebbe la memoria sulle griglie più ampie), il codice sovrascrive il valore intero della cella visitata direttamente nella struttura dati condivisa.

Durante la discesa, le celle della chiusura corrente vengono marcate con il valore costante 2. In fase di risalita la funzione le ripristina al valore 0. Questo meccanismo evita la creazione e la distruzione di oggetti a ogni livello dell'albero delle chiamate. Una marcatura costante (anziché dipendente dalla profondità) è sufficiente, perché le chiusure annidate sono disgiunte e il ripristino avviene per insieme esplicito. Inoltre, evita l'eccedenza del tipo `uint8` su catene di landmark molto lunghe.

2 Compito uno: generazione dell'ambiente

Il primo compito consiste nella stesura di un generatore procedurale per creare la griglia e posizionarvi ostacoli non attraversabili. L'obiettivo è costruire gli ambienti per il collaudo dell'algoritmo di ricerca del cammino.

2.1 Personalizzazione della griglia tramite parametri

L'utente configura il generatore attraverso i parametri della riga di comando. La griglia prodotta viene poi salvata in un file JSON che ne registra dimensioni, tipologie di ostacoli e celle non attraversabili. I parametri di configurazione includono:

- `--rows` e `--cols`: numero di righe e colonne della griglia (dimensioni $R \times C$);
- `--seed`: seme numerico per il generatore pseudo-casuale, che assicura la totale riproducibilità degli ostacoli generati nelle successive campagne di prova;
- `--types`: elenco delle tipologie di ostacoli da inserire (con la possibilità di combinarle nello scenario `mix`);
- `--density`: frazione obiettivo di celle della griglia da occupare con ostacoli fisici (compresa tra 0.0 e 1.0).

2.2 Tipologie di ostacoli e modellizzazione logica

In conformità ai requisiti per i gruppi di tre studenti, l'applicativo modella cinque forme geometriche:

1. **Semplici (simple)**: celle singole non attraversabili sparse casualmente sulla griglia. Servono a valutare il comportamento di base dell'algoritmo in ambienti a densità uniforme, senza blocchi strutturati;
2. **Agglomerati (cluster)**: gruppi compatti di celle contigue generati simulando un cammino casuale a partire da celle *seme*;

3. **Diagonali (diagonal)**: sequenze di due o tre celle adiacenti solo per uno spigolo. Questo scenario serve a valutare le regole sugli attraversamenti diagonali forzati;
4. **Recinti (enclosure)**: strutture perimetrali rettangolari dotate di area libera interna. Mettono alla prova l'algoritmo di potatura in caso di percorsi chiusi;
5. **Barre (bar)**: linee orizzontali o verticali munite di una singola apertura casuale. Costringono l'esplorazione a deviare attraverso un passaggio obbligato.

La figura 2 mostra le cinque tipologie generate isolatamente sulla stessa griglia 20×20 , in modo che ciascuna forma sia riconoscibile senza l'interferenza delle altre. Nel pannello **diagonal** si notano le coppie di celle unite solo per lo spigolo: è il caso descritto in apertura della traccia in cui l'attraversamento diagonale resta ammesso pur non essendoci un vero e proprio agglomerato di celle contigue.

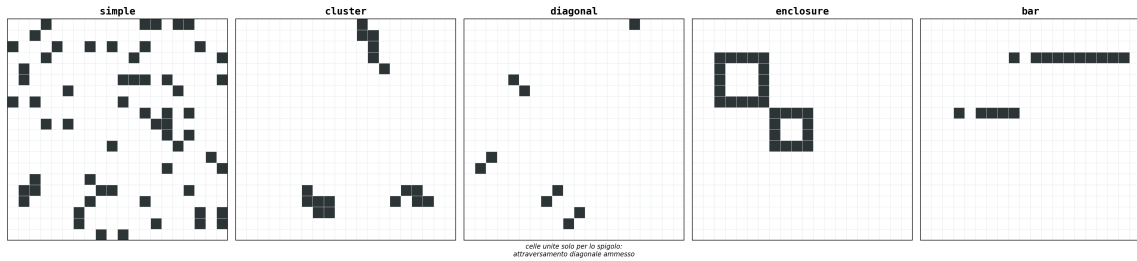


Figura 2: Le cinque tipologie di ostacolo generate isolatamente su griglie 20×20 identiche.

2.3 Algoritmo di generazione

La generazione della griglia è affidata alla classe **GridGenerator**. Per garantire l'accuratezza della densità complessiva obiettivo, il generatore inserisce prima le macrostrutture (barre, recinti, agglomerati, diagonali) e poi esegue un passo di riempimento *fine* tramite ostacoli semplici (**simple**), fino al raggiungimento esatto della quota prefissata.

I *divisori* che compaiono nello pseudocodice (10, 2,5, 16, 8) traducono la quota di celle assegnata a ciascuna macrostruttura nel numero di istanze da generare, dividendo rispettivamente per il numero medio di celle occupate da un singolo agglomerato, coppia diagonale, recinto o barra.

Di seguito si riporta lo pseudocodice di generazione:

Algorithm 1 Generazione procedurale della griglia

```

1: procedure GENERAGRIGLIA(rows, cols, types, density, seed)
2:   grid  $\leftarrow$  NuovaGriglia(rows, cols)
3:   rng  $\leftarrow$  InizializzaGeneratoreCasuale(seed)
4:   total_cells  $\leftarrow$  rows  $\times$  cols
5:   target_obstacles  $\leftarrow$  Intero(total_cells  $\times$  density)
6:
7:    $\triangleright$  Fase 1: Generazione delle macrostrutture
8:   for each type  $\in$  types  $\setminus$  {"simple"} do
9:     local_seed  $\leftarrow$  rng.GeneraIntero()
10:    if type = "cluster" then
11:      CostruisciAgglomerati(grid, (target_obstacles  $\times$  0.3)/10, local_seed)
12:    else if type = "diagonal" then
13:      CostruisciDiagonali(grid, (target_obstacles  $\times$  0.2)/2.5, local_seed)
14:    else if type = "enclosure" then
15:      CostruisciRecinti(grid, (target_obstacles  $\times$  0.2)/16, local_seed)
16:    else if type = "bar" then
17:      CostruisciBarre(grid, (target_obstacles  $\times$  0.3)/8, local_seed)
18:    end if
19:  end for
20:   $\triangleright$  Fase 2: Riempimento fine fino alla densità obiettivo
21:  current_density  $\leftarrow$  ConteggioOstacoli(grid)/total_cells
22:  if "simple"  $\in$  types or current_density < density then
23:    remaining_density  $\leftarrow$  max(0.01, density - current_density)
24:    local_seed  $\leftarrow$  rng.GeneraIntero()
25:    CostruisciSemplici(grid, remaining_density, local_seed)
26:  end if
27:  return grid
28: end procedure

```

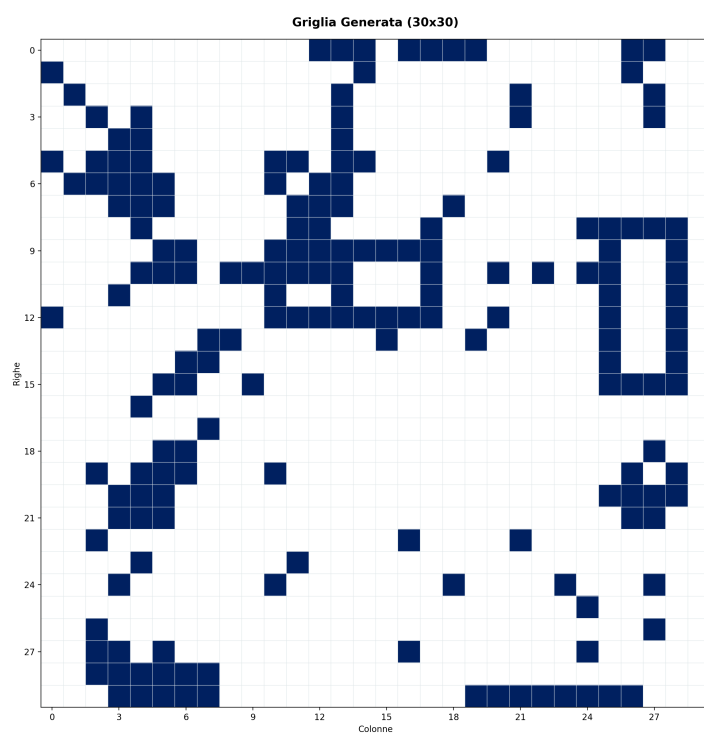


Figura 3: Griglia 30×30 prodotta dal generatore con tipologie miste (**mix**) e densità 0.2.

3 Compito due: raggiungibilità e distanza libera

Il secondo compito si concentra sull'analisi topologica dello spazio locale a partire da una cella di origine O . Vengono introdotti i concetti geometrici di distanza libera (d_{lib}), cammino libero di tipo 1 e tipo 2, contesto, complemento e frontiera della chiusura.

3.1 Distanza libera (d_{lib}) e formulazione matematica

La distanza libera (d_{lib}) rappresenta la lunghezza di un cammino libero fra O e D (se esiste), sia esso di tipo 1 o 2. Siano O e D le celle di origine e destinazione, e siano $\Delta x = |O.x - D.x|$ e $\Delta y = |O.y - D.y|$ le differenze assolute delle loro coordinate cartesiane sulla griglia.

Definiamo $\Delta_{\min} = \min(\Delta x, \Delta y)$ come il numero massimo di passi diagonali puri eseguibili e $\Delta_{\max} = \max(\Delta x, \Delta y)$ come la dimensione dominante dello spostamento. La formula matematica esatta per il calcolo di $d_{\text{lib}}(O, D)$ è:

$$d_{\text{lib}}(O, D) = \sqrt{2}\Delta_{\min} + (\Delta_{\max} - \Delta_{\min}) \quad (1)$$

Questa formula di costo $O(1)$ fornisce il limite inferiore richiesto dalla potatura geometrica descritta di seguito.

Fra due celle O e D reciprocamente raggiungibili senza ostacoli interposti possono esistere fino a due cammini liberi distinti, come illustrato in figura 4: il tipo 1 percorre prima la diagonale e poi il tratto cardinale residuo, il tipo 2 nell'ordine inverso. Entrambi hanno lunghezza $d_{\text{lib}}(O, D)$, poiché in una griglia 8-connesse l'ordine in cui si eseguono i passi diagonali e quelli cardinali non ne altera il numero complessivo.

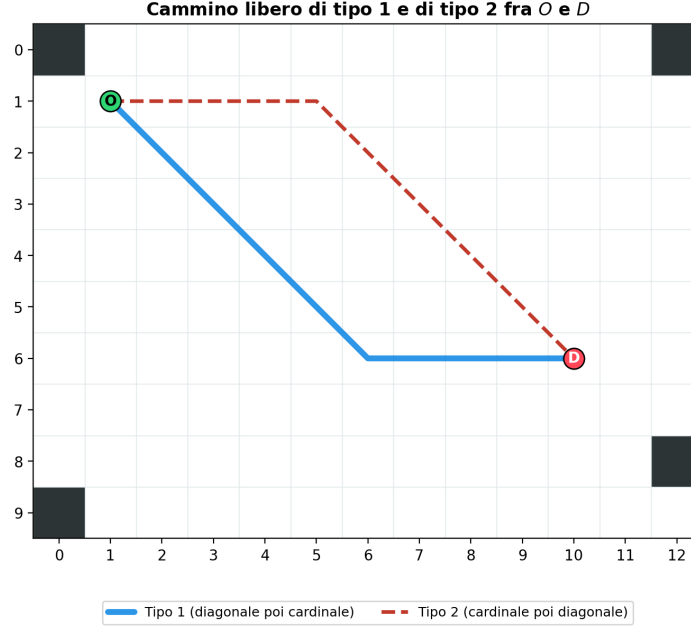


Figura 4: I due cammini liberi possibili fra O e D : tipo 1 (diagonale poi cardinale) e tipo 2 (cardinale poi diagonale).

3.2 Contesto e complemento

La raggiungibilità locale tramite cammini liberi definisce due insiemi disgiunti di celle all'interno dello spazio libero:

- **Contesto:** l'insieme delle celle raggiungibili da O tramite un cammino libero di tipo 1, definito dalla sequenza prima mosse diagonali oblique, poi mosse cardinali residue, oppure da cammini composti esclusivamente da un tipo di mossa;
- **Complemento:** l'insieme delle celle raggiungibili da O solo tramite un cammino libero di tipo 2, definito dalla sequenza inversa prima mosse cardinali, poi mosse diagonali residue. Le celle che ammettono entrambi i tipi di cammino appartengono per convenzione al solo contesto.

L'unione di contesto e complemento forma la **chiusura** di O .

3.3 Proiezione di raggi geometrici ($O(\max(R, C)^2)$)

Per determinare gli insiemi contesto e complemento, il programma implementa la **proiezione di raggi**. L'analisi controlla le linee di raggiungibilità procedendo a raggiera a partire dall'origine, riducendo la complessità da quadratica ($O(R^2 \cdot C^2)$ per una ricerca cella per cella) a $O(\max(R, C)^2)$.

L'algoritmo per il **contesto (tipo 1)** opera come segue:

1. **Proiezione cardinale:** vengono tracciati i 4 raggi cardinali puri (Nord, Sud, Est, Ovest) a partire dall'origine O , arrestandosi non appena si incontra una cella ostacolo;
2. **Proiezione diagonale ed espansione a gomito:** per ciascuno dei 4 quadranti del piano cartesiano locale:
 - Si proietta il raggio diagonale puro (NE, NO, SE, SO);
 - Per ogni cella attraversabile raggiunta lungo questo raggio diagonale, si avvia un'espansione cardinale rettilinea parallela agli assi (espansione a gomito), che procede verso l'esterno fino a incontrare un ostacolo o il bordo della griglia.

L'algoritmo per il **complemento (tipo 2)** inverte simmetricamente le fasi di proiezione: per ciascuno dei 4 quadranti si proiettano prima i raggi cardinali e, da ogni cella libera raggiunta su di essi, si espandono raggi in diagonale pura. Al termine si escludono le celle già presenti nel contesto.

Questo accorgimento analizza ogni cella limitando le operazioni di lettura sulla matrice.

Di seguito si riporta lo pseudocodice dei due algoritmi. La scrittura $P + c$ indica la cella ottenuta spostando P di un passo lungo la direzione c . Una cella si dice *libera* se è interna alla griglia e non appartiene a *osta*.

Algorithm 2 Calcolo del contesto (celle raggiungibili con cammini liberi di tipo 1)

```
1: procedure CALCOLACONTESTO( $O, dim, osta$ )
2:    $contesto \leftarrow \{O\}$ 
3:   for each direzione cardinale  $c \in \{\text{Nord, Sud, Est, Ovest}\}$  do
4:      $P \leftarrow O + c$ 
5:     while  $P$  è libera do
6:        $contesto \leftarrow contesto \cup \{P\}$ 
7:        $P \leftarrow P + c$ 
8:     end while
9:   end for
10:  for each direzione diagonale  $d \in \{\text{NE, NO, SE, SO}\}$  do
11:     $P \leftarrow O + d$ 
12:    while  $P$  è libera do
13:       $contesto \leftarrow contesto \cup \{P\}$ 
14:       $\triangleright$  Espansione cardinale orizzontale e verticale a partire da  $P$ 
15:      for each  $c \in \{\text{componente orizzontale di } d, \text{componente verticale di } d\}$ 
16:        do
17:           $Q \leftarrow P + c$ 
18:          while  $Q$  è libera do
19:             $contesto \leftarrow contesto \cup \{Q\}$ 
20:             $Q \leftarrow Q + c$ 
21:          end while
22:        end for
23:       $P \leftarrow P + d$ 
24:    end while
25:  end for
26:  return  $contesto$ 
27: end procedure
```

Algorithm 3 Calcolo del complemento (celle raggiungibili solo con cammini liberi di tipo 2)

```

1: procedure CALCOLACOMPLEMENTO( $O, dim, osta, contesto$ )
2:    $complemento \leftarrow \emptyset$ 
3:   for each direzione diagonale  $d \in \{NE, NO, SE, SO\}$  do
4:     for each  $c \in \{\text{componente orizzontale di } d, \text{componente verticale di } d\}$ 
       do
5:        $P \leftarrow O + c$ 
6:       while  $P$  è libera do
7:          $\triangleright$  Espansione diagonale a partire dalla cella  $P$  del raggio cardinale
8:          $Q \leftarrow P + d$ 
9:         while  $Q$  è libera do
10:          if  $Q \notin contesto$  then
11:             $complemento \leftarrow complemento \cup \{Q\}$ 
12:          end if
13:           $Q \leftarrow Q + d$ 
14:        end while
15:         $P \leftarrow P + c$ 
16:      end while
17:    end for
18:  end for
19:  return  $complemento$ 
20: end procedure

```

3.4 Frontiera della chiusura

La **frontiera** di una cella O è l'insieme di tutte le celle della chiusura di O che sono adiacenti a una cella attraversabile che non appartiene a tale chiusura. Formalmente, la frontiera è definita come segue:

$$\begin{aligned}
 \text{Frontiera}(O) = \{c \in \text{Chiusura}(O) \mid \exists v \in \text{Vicini}(c) \\
 \text{t.c. } \text{Stato}(v) = 0 \wedge v \notin \text{Chiusura}(O)\}
 \end{aligned}
 \tag{2}$$

Il codice estrae la frontiera limitando l'ispezione al vicinato delle sole celle incluse

nella chiusura. Ad ogni cella di frontiera viene associato il tipo del cammino libero da cui proviene (1 per il contesto, 2 per il complemento).

La figura 5 illustra un esempio concreto su una griglia 15×15 , in cui due ostacoli spezzano parzialmente due quadranti dell'origine O . Nella parte sinistra sono evidenziati il contesto (in verde) e il complemento (in arancio), sovrapposti agli otto raggi primari (4 cardinali e 4 diagonali) tracciati dall'algoritmo. Questo tracciamento costituisce il passo geometrico che precede le espansioni a gomito necessarie a riempire il resto della chiusura. Nella parte destra, infine, è visibile la frontiera della loro unione (cerchiata in rosso).

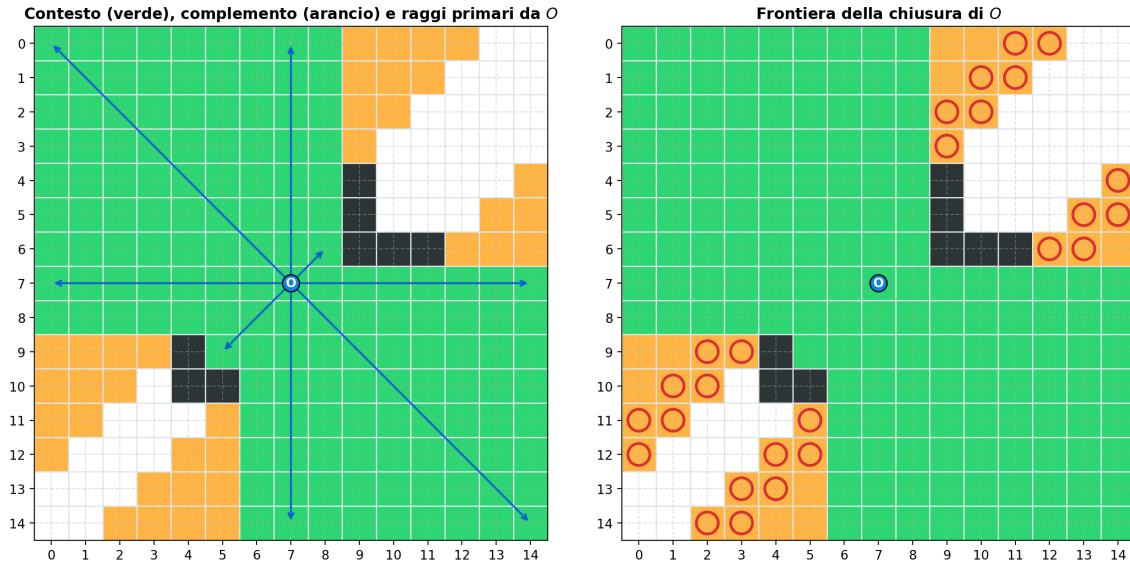


Figura 5: Contesto, complemento e frontiera calcolati tramite proiezione di raggi su una griglia dimostrativa.

4 Compito tre: ricerca del cammino minimo

Il terzo compito descrive la progettazione e l'implementazione dell'algoritmo ricorsivo CAMMINOMIN per la ricerca del cammino minimo globale tra una cella di origine O e una cella di destinazione D . L'algoritmo opera su un grafo implicito i cui nodi sono costituiti dalle celle della griglia e i cui archi rappresentano i cammini liberi.

4.1 Definizione ricorsiva e flusso esecutivo

L'algoritmo si basa sulla scomposizione del percorso ottimale in una successione di sotto-cammini liberi. A ogni livello di ricorsione:

1. **Verifica del caso base:** si calcola la chiusura dell'origine corrente. Se la destinazione D appartiene a tale chiusura (ossia è raggiungibile con un singolo cammino libero di tipo 1 o tipo 2), la ricorsione si interrompe immediatamente ritornando la distanza ideale $d_{\text{lib}}(O, D)$ e i landmark associati;
2. **Espansione della frontiera:** se D non è direttamente raggiungibile, si estrae la frontiera locale della chiusura. Se la frontiera è vuota, il ramo corrente rappresenta un vicolo cieco. Viene quindi ritornato un costo infinito e una sequenza vuota di landmark;
3. **Esplorazione e ritracciamento:** per ciascun candidato di frontiera F si calcola il costo parziale $l_F = d_{\text{lib}}(O, F)$. Se la mossa supera le condizioni di potatura, si effettua una chiamata ricorsiva con origine in F e destinazione D , aggiornando dinamicamente lo stato della griglia.

Le chiusure calcolate nei livelli superiori dell'albero di ricorsione vengono marcate come ostacoli temporanei, per prevenire cicli e per evitare di riesplorare aree già visitate.

4.2 Analisi della potatura: riga 16 contro riga 17

Per ridurre lo spazio di ricerca e per evitare l'esplorazione di rami non ottimali, la specifica introduce due condizioni di potatura alternative:

- **Potatura debole (riga 16):** la cella F viene esplorata solo se la distanza ideale dal nodo corrente è inferiore al cammino ottimale globale provvisorio trovato finora:

$$l_F < \text{lunghezzaMin} \quad (3)$$

- **Potatura forte (riga 17):** sfrutta la stima ammissibile della distanza libera residua da F alla destinazione D , potando il ramo se la somma del cammino parziale e della stima minima rimanente non può battere l'ottimo corrente:

$$l_F + d_{\text{lib}}(F, D) < \text{lunghezzaMin} \quad (4)$$

La potatura forte è più onerosa per via del calcolo aggiuntivo di d_{lib} , ma riduce i nodi visitati in presenza di ostacoli complessi.

4.3 Potatura branch-and-bound globale

Lo pseudocodice valuta la condizione di riga 16/17 rispetto a `lunghezzaMin`, cioè il minimo *locale* della singola invocazione. L'implementazione la rafforza confrontandola con un **limite superiore globale** L_{glob} , condiviso da tutte le invocazioni ricorsive e aggiornato ogni volta che si trova un cammino completo più breve. Un candidato di frontiera F viene quindi esplorato solo se

$$\ell_{\text{acc}} + l_F + h(F) < L_{\text{glob}} \quad (5)$$

dove ℓ_{acc} è la lunghezza già accumulata dall'origine principale e $h(F)$ vale 0 nella potatura debole (riga 16) e $d_{\text{lib}}(F, D)$ in quella forte (riga 17). Poiché d_{lib} **non sovrastima mai la distanza reale**, la stima è ammissibile e la potatura non scarta mai un cammino ottimo: si tratta di una potatura branch-and-bound. Finché non si è trovato il primo cammino completo, L_{glob} vale $+\infty$ e non si pota nulla. La potatura non richiede alcuna passata preliminare: il limite emerge dalla ricerca stessa.

Per produrre questo confronto, `camminomin()` accetta un parametro opzionale `visited_closures`: se fornito, ogni chiamata ricorsiva vi accumula le celle della

propria chiusura, tracciando così l'impronta complessiva della ricerca a fini di analisi. Il parametro non altera in alcun modo la logica dell'algoritmo, non è utilizzato quando omesso (valore predefinito `None`) ed è impiegato solo dallo script che genera la figura seguente.

Per rendere tangibile l'effetto della potatura, la figura 6 confronta, sulla stessa coppia origine–destinazione di una griglia 18×18 con ostacoli ad agglomerati, l'unione di tutte le chiusure esplorate durante l'intera ricerca nei due casi. La potatura debole (riga 16) allaga quasi l'intero spazio libero raggiungibile (260 celle su 324), perché continua a inseguire rami la cui unica colpa è avere percorso finora più strada, anche quando sono geometricamente lontani dalla destinazione. La potatura forte (riga 17) traccia invece un corridoio molto più mirato verso D (97 celle), poiché la stima $d_{\text{lib}}(F, D)$ scarta subito i rami che, nel migliore dei casi, non potrebbero comunque competere con il minimo corrente. Su questa istanza il tempo di calcolo passa da 21 ms a meno di un millisecondo.

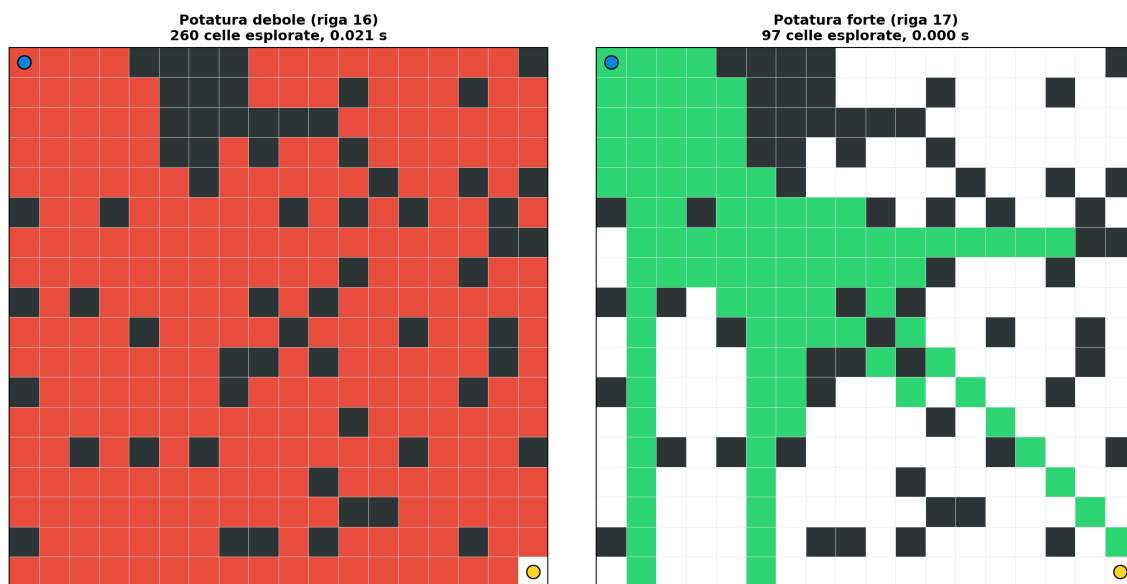


Figura 6: Impronta di esplorazione (unione delle chiusure visitate) a confronto fra potatura debole e forte, stessa coppia origine–destinazione.

4.4 Controllo preliminare di raggiungibilità con i componenti connessi

La ricerca ricorsiva ha un punto debole strutturale sulle coppie irraggiungibili: quando la destinazione si trova in una regione isolata da ostacoli (per esempio all'interno di un recinto chiuso), l'algoritmo non ha modo di accorgersene se non esaurendo l'intera frontiera di ogni ramo, con un costo che cresce in modo esponenziale prima di poter rispondere che il cammino non esiste.

Per eliminare questo caso patologico l'implementazione esegue, soltanto alla chiamata radice, un riempimento a inondazione iterativo (visita in ampiezza con una coda, funzione `compute_components()`) che etichetta ogni cella libera con l'identificativo del proprio componente connesso rispetto al vicinato a otto celle, in tempo e spazio $O(R \cdot C)$. Se origine e destinazione ricadono in componenti diversi, la funzione ritorna subito distanza infinita senza avviare alcuna ricorsione. La correttezza discende dalla specifica stessa (diapositiva 3 del testo dell'elaborato): il passo diagonale fra due celle libere è sempre ammesso, anche quando due ostacoli si toccano per lo spigolo, quindi la connettività a otto coincide esattamente con la raggiungibilità e il controllo non può mai dichiarare irraggiungibile una coppia che non lo sia. L'esito del calcolo resta identico in entrambi i casi (distanza infinita); cambia soltanto il costo per arrivarci.

L'effetto è quantificato da un esperimento dedicato (`scripts/verifica_componenti.py`): su una griglia 50×50 con densità 0.2 e un recinto chiuso, con destinazione interna al recinto e origine esterna, il controllo preliminare risponde in 0,014 secondi, mentre la stessa interrogazione senza controllo raggiunge il tempo limite di 600 secondi dopo quasi 13,8 milioni di invocazioni ricorsive senza essere riuscita a dimostrare l'irraggiungibilità. Il controllo è attivo in modo predefinito e disattivabile dalla riga di comando con `--no-component-check` per riprodurre il confronto. Va osservato che il beneficio riguarda soltanto le coppie realmente disconnesse: sulle coppie raggiungibili il controllo aggiunge un costo lineare trascurabile e la ricerca procede come prima.

4.5 Ottimizzazioni fisiche e implementative

L'implementazione prevede anche le seguenti ottimizzazioni strutturali:

- **Ordinamento euristico della frontiera:** prima di scorrere il ciclo sulle celle della frontiera, queste vengono ordinate per valori crescenti della distanza libera dalla destinazione tramite l'ordinamento nativo di Python (Timsort, $O(n \log n)$) [2]. Questa strategia esplora per primi i percorsi più promettenti, riducendo il numero di rami da visitare prima di aggiornare il minimo globale;
- **Ritracciamento sul posto:** realizzato marcando le celle della chiusura corrente nella matrice condivisa con un valore costante (ostacolo temporaneo) ed eseguendo il ripristino in tempo costante durante la risalita, senza allocazione di memoria.

Un tentativo iniziale aveva introdotto anche la memoizzazione dei sotto-problemi, con chiave data da origine, destinazione e insieme delle celle temporaneamente bloccate. Il tentativo ha rivelato però un difetto logico: il risultato di ogni sotto-problema dipende, attraverso la potatura globale, dalla lunghezza accumulata delle chiamate esterne, che non fa parte della chiave. Un risultato memorizzato con una lunghezza accumulata elevata, e quindi con i rami ottimali già potati, poteva essere reimpiegato erroneamente in una chiamata con lunghezza accumulata minore, producendo una risposta subottimale e violando la simmetria richiesta dalla specifica (diapositiva 64 del testo dell'elaborato). La memoizzazione è stata pertanto rimossa, mentre le prestazioni restano garantite dalla potatura branch-and-bound globale e dall'ordinamento euristico della frontiera.

Tutte queste ottimizzazioni sono implementate come parametri booleani opzionali di `camminomin()` (`use_strong_pruning`, `randomize_frontier`).

4.6 Requisiti funzionali implementati

Il software prevede le seguenti funzionalità richieste dalla specifica:

- **Ricostruzione del cammino fisico (`reconstruct_path`):** a partire dalla sequenza di landmark ritornata $((O, 0), (F_1, t_1), \dots, (D, t_D))$, il sistema ricostruisce l'esatta sequenza continua di celle attraversabili (mosse cardinali e diagonali) unendo i singoli tratti liberi di tipo 1 o tipo 2 e validandone l'attraversabilità geometrica;
- **Gestione dell'interruzione per superamento del tempo limite:** l'ispezione del tempo trascorso a ogni chiamata ricorsiva, consente di arrestare la ricerca in modo sicuro se si supera il tempo limite programmato dall'utente; salvando e restituendo la sequenza di celle di lunghezza minore individuata fino a quel momento.

La figura 7 mostra un primo esempio didattico di cammino minimo ricostruito: in azzurro trasparente il percorso fisico completo, con la spezzata e i marcatori romboidali che ne evidenziano la sequenza di landmark (visualizzazione ibrida, diapositiva 53 del testo dell'elaborato). Su questa istanza la destinazione si trova quasi in linea diagonale con l'origine, per cui la sequenza si riduce a un solo landmark intermedio.

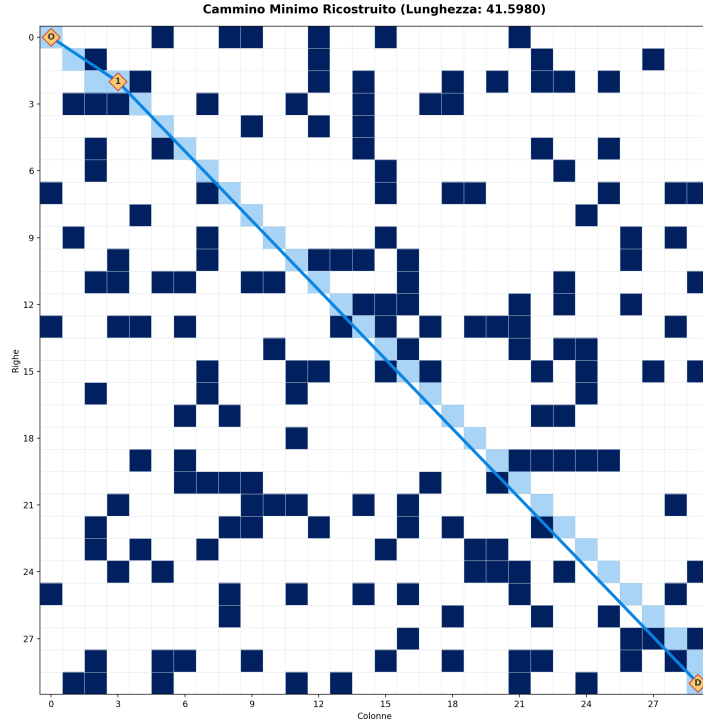


Figura 7: Cammino minimo ricostruito su un'istanza semplice: pochi ostacoli, sequenza breve di landmark.

La figura 8 riporta invece un caso più rappresentativo del comportamento dell'algoritmo su griglie realistiche: una griglia 40×40 con agglomerati, barre e recinti (densità 0.25), dove il cammino minimo fra i due angoli opposti richiede 7 landmark intermedi per aggirare gli ostacoli via via incontrati. Ogni deviazione visibile nella spezzata corrisponde a una cella di frontiera da cui l'algoritmo ha aperto un nuovo cammino libero verso la destinazione.

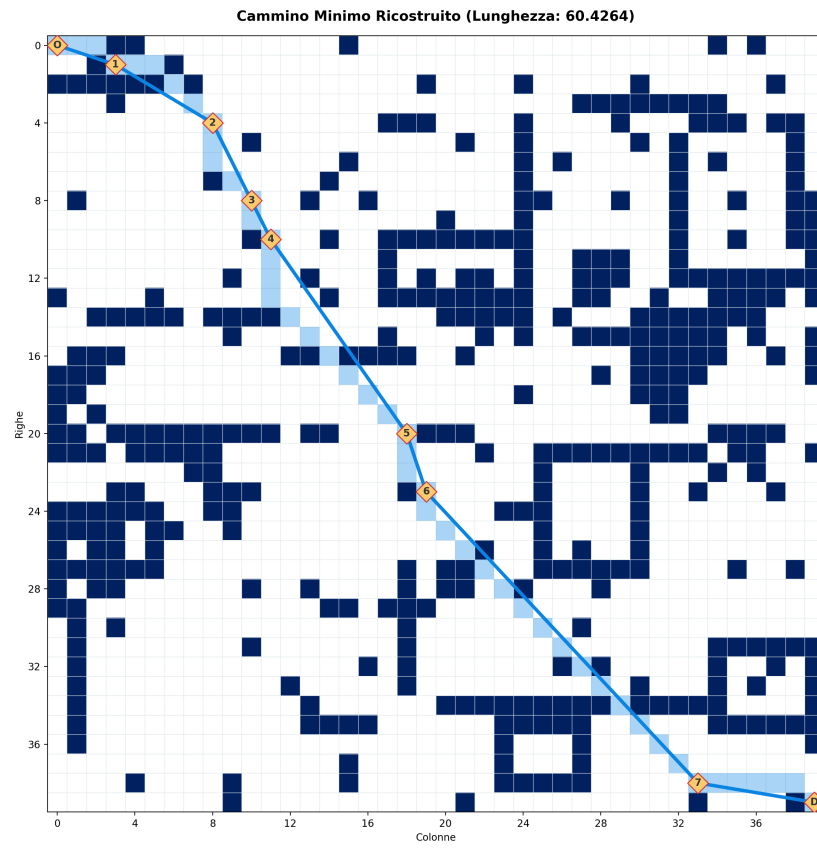


Figura 8: Cammino minimo su una griglia 40×40 densamente ostacolata: 7 landmark intermedi.

5 Compito quattro: sperimentazione e analisi

Il quarto compito valuta le prestazioni del sistema. La sezione presenta le analisi di complessità spaziale e temporale, seguite dai risultati empirici.

5.1 Analisi di complessità spaziale asintotica

La complessità spaziale (occupazione di memoria centrale) è dettata dalla pila delle chiamate ricorsive e dalle strutture dati locali allocate a ogni livello:

- **Pila di ricorsione:** sia l la profondità massima della ricorsione, corrispondente al numero di landmark intermedi. Nel caso peggiore l è limitato dal numero totale di celle della griglia, quindi $l \leq R \times C$. Lo spazio della pila è $O(l)$;
- **Strutture locali di chiusura e frontiera:** a ogni livello ricorsivo il calcolo della chiusura corrente occupa al più $O(R \times C)$ celle. Lo stesso vale per la lista delle celle di frontiera estratte;
- **Gestione dello stato della griglia:** molti approcci semplici duplicano l'intero stato della griglia a ogni chiamata per simulare i nuovi ostacoli temporanei, portando la complessità spaziale totale a $O(R \times C \cdot l)$.

Grazie all'ottimizzazione del ritracciamento sul posto su array NumPy, l'implementazione mantiene un'unica griglia condivisa. Le chiusure temporanee vengono marcate tramite sovrascritture. Lo spazio di memoria allocato è legato alle variabili locali della ricorsione, contenendo l'occupazione reale.

5.2 Analisi di complessità temporale asintotica

Analisi della complessità temporale:

- **Caso peggiore (griglie complesse e ostruite):** in assenza di potature l'algoritmo esplora combinatoriamente i rami di frontiera con complessità $O(K^l)$, dove K è la dimensione della frontiera. La potatura forte e l'ordinamento euristico riducono il fattore di ramificazione, come mostrano i dati sperimentali.

Sulle istanze provate il loro effetto è netto, pur non cambiando l'andamento asintotico;

- **Caso migliore (destinazione nella chiusura iniziale):** se la destinazione D appartiene al contesto o al complemento dell'origine O , l'algoritmo si arresta immediatamente al livello ricorsivo 0 (caso base). Il tempo di esecuzione è interamente dominato dal calcolo della chiusura iniziale tramite proiezione di raggi, con una complessità pari a:

$$T_{\text{best}} = O(\max(R, C)^2) \quad (6)$$

Questo si traduce in tempi di elaborazione inferiori al millisecondo anche per griglie di lato elevato (su lato 10 il calcolo richiede circa 0,3 ms).

5.3 Nota metodologica sulla misura

I tempi riportati nei grafici sono cronometrati in esecuzioni prive di profilazione della memoria. La misura del picco di memoria, infatti, introduce un sovraccarico di circa cinque volte: mantenerla attiva durante il cronometraggio falserebbe i tempi (sulla griglia 50×50 la stessa esecuzione passava da circa 1,7 s a circa 9,6 s). L'occupazione di memoria viene perciò rilevata in una passata separata e distinta da quella temporale.

5.4 Analisi dei risultati sperimentali e discussione dei grafici

La campagna sperimentale è stata eseguita su griglie da 10×10 fino a 200×200 , variando la densità degli ostacoli e analizzando le diverse strategie. Per le campagne su densità, potatura e ordinamento, ogni configurazione è stata provata su più coppie origine–destinazione (la coppia d'angolo, geometricamente più impegnativa, e quattro coppie casuali di celle attraversabili); le metriche riportate sono le mediane dei campioni raccolti. Come richiesto dalla specifica (diapositiva 64 del testo dell'elaborato), ciascuna coppia distinta impiegata nella campagna riceve la doppia

invocazione con i parametri scambiati ($O \rightarrow D$ e $D \rightarrow O$), eseguita nella configurazione a potatura forte. La simmetria è una proprietà dell'algoritmo per una data coppia (griglia, O , D) e non dipende dalla strategia di potatura né dall'ordine di visita della frontiera, poiché entrambe convergono comunque all'ottimo globale grazie all'ammissibilità della potatura (si veda il Compito 3): verificarla una volta per coppia è quindi sufficiente. I campioni a potatura debole e a ordinamento casuale, calcolati sulle stesse coppie già verificate, servono unicamente al confronto prestazionale discusso più avanti. L'esito complessivo della verifica è registrato nel file `simmetria_campagna.json`. I risultati sono riassunti in sei grafici, discussi di seguito.

5.4.1 1. Tempo in funzione della densità di ostacoli

Il file `1_time_vs_density.png` illustra l'andamento del tempo di esecuzione al variare della densità degli ostacoli da 0.05 a 0.45, ripetuto su due taglie di griglia (50×50 e 100×100) per verificare se una griglia più grande renda più marcata la transizione di fase. Su 50×50 il tempo mediano resta sotto i 10 ms fino a densità 0.35 e sale a circa 1,3 s al 45%: la scala rimane quella dei millisecondi/secondi. Su 100×100 , invece, il tempo mediano supera già il secondo a densità 0.25 e 0.35, raggiungendo il tempo limite di 20 s (la ricerca non completa più della metà dei campioni entro quel budget), per poi ridiscendere a circa 8 s al 45%. Raddoppiare il lato della griglia sposta quindi la transizione di fase da una scala di millisecondi a una di decine di secondi, confermando che la sola dimensione del problema, a parità di densità relativa, aggrava sensibilmente la difficoltà di ricerca. Questo è il fianco crescente della **transizione di fase** tipica dei problemi di ricerca:

- A densità basse lo spazio è aperto e la destinazione ricade spesso nella chiusura iniziale dell'origine, per cui il cammino viene trovato quasi istantaneamente con pochi landmark, indipendentemente dalla taglia;
- Avvicinandosi alla soglia critica lo spazio si frammenta: restano cammini tortuosi, ma difficili da isolare, e l'algoritmo deve esplorare rami profondi prima di convergere. Su una griglia più grande questi rami sono più numerosi e più

lunghi, da cui il salto di quasi tre ordini di grandezza osservato passando da 50×50 a 100×100 alla stessa densità;

- Oltre la soglia di connettività (densità non incluse nelle prove) ci si attenderebbe un nuovo calo, perché molte coppie diventano irraggiungibili e l'algoritmo lo rileva subito potando l'intera frontiera; il calo già visibile al 45% su 100×100 è coerente con questa attesa.

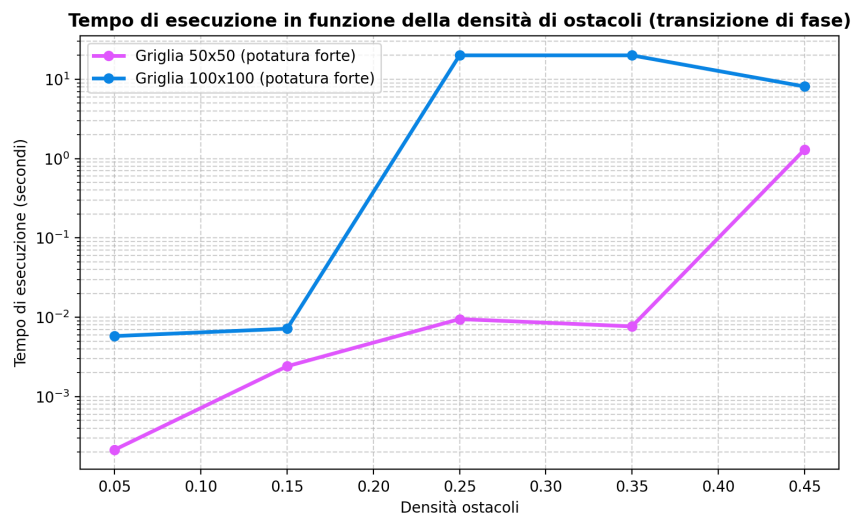


Figura 9: Tempo di esecuzione in funzione della densità di ostacoli, a confronto fra due taglie di griglia (transizione di fase).

5.4.2 2. Confronto delle configurazioni (potatura e ordinamento)

Il diagramma a barre in `2_pruning_comparison_boxplot.png` (in scala logaritmica) confronta, sulla coppia d'angolo di una griglia 50×50 e per i diversi tipi di ostacolo, tre configurazioni realmente distinte: potatura debole con ordinamento euristico, potatura forte con ordinamento euristico e potatura forte con ordinamento casuale. Non esiste una quarta barra isolata per il solo ordinamento euristico, perché nella campagna quest'ultimo viene sempre misurato insieme alla potatura forte, che ne costituisce anche il termine di paragone per il confronto con l'ordinamento casuale.

"Raggiungere il tempo limite" indica qui che la ricerca è stata interrotta a 20 s senza completarsi: il tempo mostrato in grafico per quei casi è quindi convenzionalmente il tempo limite stesso, non un tempo di risoluzione effettiva (lo stesso significato di `timed_out` già introdotto nel Compito 3). Entrambe le scelte progettuali risultano decisive:

- La potatura forte (riga 17) risolve ogni scenario in pochi millisecondi (da 0,006 a 0,014 s), mentre la potatura debole (riga 16) raggiunge il tempo limite di 20 s su quasi tutti i tipi, cioè non completa la ricerca in quel tempo (fa eccezione il tipo `bar`, risolto in circa 0,5 s). La stima ammissibile $d_{\text{lib}}(F, D)$ aggiunta dalla riga 17 sfoltisce l'albero di ricerca in misura molto maggiore;
- L'ordinamento euristico della frontiera (per d_{lib} crescente dalla destinazione) è altrettanto importante: con l'ordinamento casuale la ricerca raggiunge il tempo limite su quattro tipi su cinque, mentre con quello euristico si conclude in millisecondi. Esplorare per primi i candidati più vicini alla destinazione consente di trovare presto un buon limite superiore, di cui la potatura branch-and-bound beneficia subito.

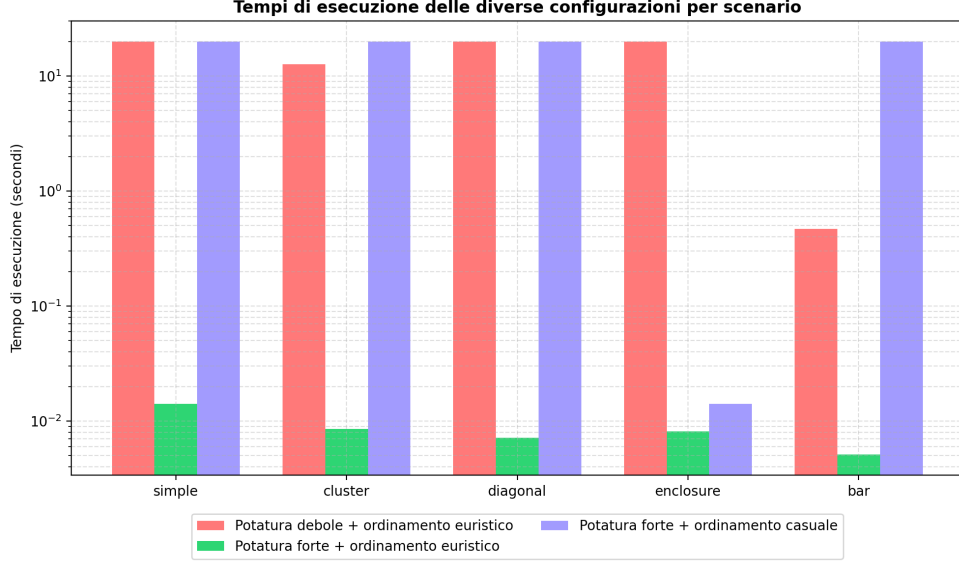


Figura 10: Tempi di esecuzione delle tre configurazioni distinte (potatura debole/forte, ordinamento euristico/casuale) per tipologia di ostacolo.

Questo confronto è condotto su griglie 50×50 , la stessa taglia della campagna sulla densità. Uno script supplementare (`scripts/verifica_potatura_100.py`) ripete il confronto debole/forte su un singolo scenario rappresentativo (`cluster`) alla taglia 100×100 : a questa dimensione, con un tempo limite di 60 s, *nemmeno la potatura forte* completa più la ricerca (circa 666 000 chiamate ricorsive contro le circa 2,67 milioni della debole nello stesso tempo). Il vantaggio della potatura forte resta netto in termini di nodi esplorati a parità di tempo, ma a 100×100 non è più sufficiente da solo a garantire una risposta entro un tempo limite dell'ordine dei secondi, anticipando quanto osservato più sistematicamente nella campagna di scaling (§5.4.3).

5.4.3 3. Crescita temporale asintotica (scala bilogaritmica)

Il grafico in `3_complexity_scaling_loglog.png` riporta i tempi sulla coppia d'angolo al crescere del lato (fino a 200×200), per la potatura debole e per quella forte. Il tempo limite di questa campagna è stato fissato a 600 s (10 minuti), dieci volte

quello inizialmente usato, per accertare se il muro osservato da lato 100 in su fosse un limite strutturale dell'algoritmo oppure un semplice artefatto di un budget di tempo insufficiente. Si osserva che:

1. La **potatura forte** scala molto meglio alle taglie piccole: risolve il lato 10 e 20 in una frazione di millisecondo e il lato 50 in circa 1,6 s, mentre la **potatura debole** richiede già 1,7 s al lato 20 e non completa più la ricerca al lato 50, esaurendo l'intero budget di 600 s;
2. Da lato 100 in poi **nessuna delle due configurazioni completa la ricerca**, nemmeno con il tempo decuplicato: entrambe esauriscono i 600 s a 100, 150 e 200. Al lato 200, in particolare, né la potatura debole né quella forte trovano un cammino completo qualsiasi entro il tempo concesso (lunghezza restituita infinita per entrambe), a differenza delle taglie minori dove un cammino, anche se non necessariamente ottimo, veniva comunque individuato;
3. La potatura forte continua comunque a esplorare meno nodi a parità di tempo scaduto: a lato 100 arresta la ricerca dopo circa 10,3 milioni di chiamate ricorsive contro i 20,6 milioni della debole, e a lato 150 circa 13,2 milioni contro 18,4 milioni. A lato 200 il vantaggio si annulla (circa 53,5 milioni contro 53,1 milioni): a queste dimensioni entrambe le configurazioni saturano ugualmente lo spazio di ricerca esplorabile in 600 s, segno che il fattore di ramificazione della frontiera è ormai troppo ampio perché la sola potatura, per quanto efficace, possa contenerlo;
4. Il limite osservato è quindi strutturale, non un artefatto del tempo limite: aumentare il budget di un ordine di grandezza (da 60 a 600 s) non ha spostato il muro oltre il lato 50-100. Questo conferma quanto già argomentato nell'analisi di complessità: la potatura riduce la base della crescita esponenziale $O(K^l)$, non il suo esponente, che continua a dipendere dal numero di landmark l e dalla dimensione tipica K della frontiera, entrambi crescenti con il lato della griglia.

In tutti i casi che superano il tempo limite l’algoritmo restituisce comunque il miglior cammino individuato fino a quel momento (se esistente), segnalando che il calcolo non è stato completato (requisito funzionale della diapositiva 71 del testo dell’elaborato).

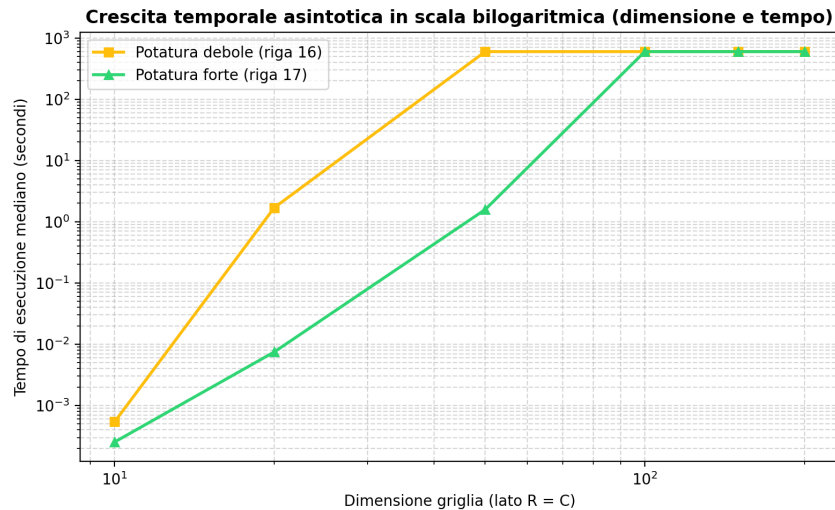


Figura 11: Crescita del tempo di esecuzione in scala bilogaritmica al crescere del lato della griglia.

5.4.4 4. Correlazione tra tempo e celle di frontiera

Il diagramma a dispersione in `4_scatter_time_vs_frontier.png` (scala bilogaritmica) mappa ogni prova ponendo sulle ascisse le celle di frontiera totali considerate e sulle ordinate il tempo di calcolo. I punti sono fortemente allineati: il tempo di esecuzione cresce in modo regolare con il volume di frontiera analizzato. La pendenza della retta di regressione è circa 0,82, leggermente sotto-lineare, coerente con un costo per singola cella di frontiera che non cresce con la dimensione del problema, grazie alla rappresentazione compatta su matrice NumPy.

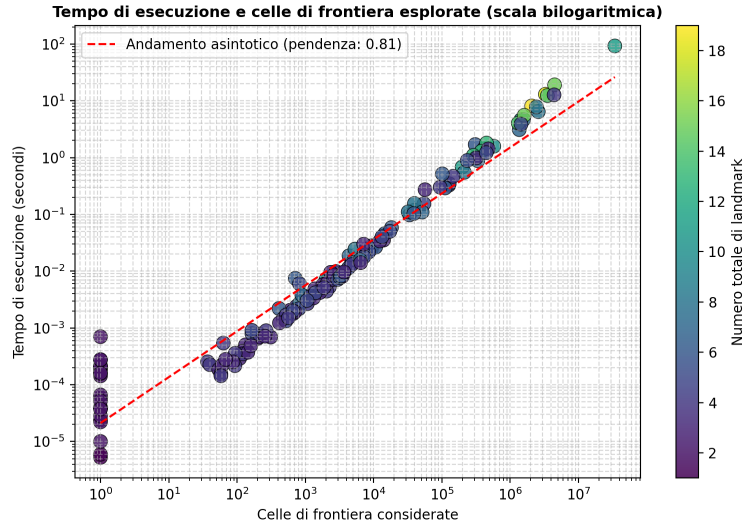


Figura 12: Dispersione tempo di esecuzione contro celle di frontiera considerate, con retta di regressione in scala logaritmica.

5.4.5 5. Correlazione tra tempo e invocazioni ricorsive

Il grafico in `5_scatter_time_vs_recursion.png` mappa il tempo in funzione delle chiamate ricorsive totali, colorando i punti in base alla lunghezza del cammino. L'allineamento è marcato (pendenza di regressione circa 0,94): il numero di chiamate ricorsive è il fattore primario del costo temporale, a conferma del fatto che contenere la pila delle chiamate è la leva principale sulle prestazioni.

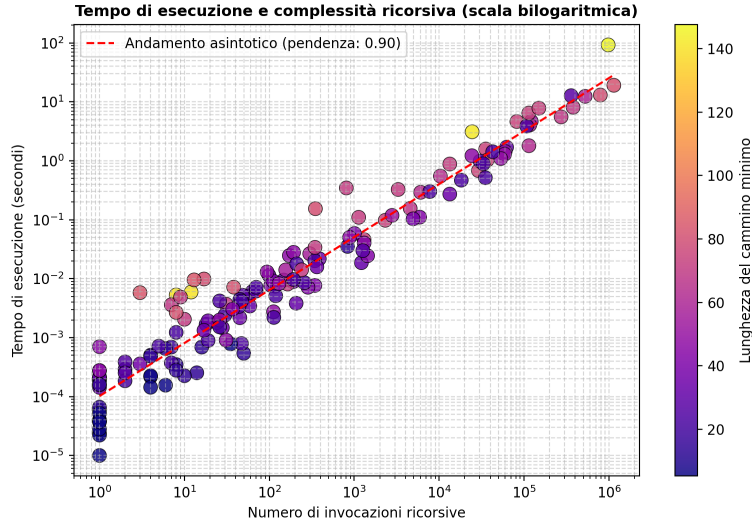


Figura 13: Dispersione tempo di esecuzione contro numero di invocazioni ricorsive, colorata per lunghezza del cammino.

5.4.6 6. Verifica formale della simmetria

Il grafico a barre in `6_symmetry_per_type.png` mostra l'esito delle prove di simmetria ($O \rightarrow D$ contro $D \rightarrow O$) eseguite su cinque coppie casuali per ciascuna tipologia di ostacolo: tutte e cinque le coppie risultano verificate per ognuno dei cinque tipi, senza fallimenti né esclusioni. A queste prove dedicate si aggiunge la doppia invocazione effettuata su ciascuna delle coppie distinte utilizzate nelle campagne di scalabilità, densità (sulle due taglie) e potatura (si veda l'introduzione di questa sezione): su 81 coppie totali, 62 sono risultate verificabili (le altre 19 escluse perché una delle due direzioni ha raggiunto il tempo limite, un'esclusione più frequente rispetto alla campagna precedente proprio a causa del tempo limite decuplicato sullo scaling e della nuova taglia 100×100 nella campagna di densità) e nessuna ha prodotto incongruenze metriche. Si registra quindi il 100% di successi sulle coppie verificabili, a validazione sperimentale dell'esattezza dell'implementazione, come richiesto esplicitamente dalla specifica (diapositive 63–64 del testo dell'elaborato).

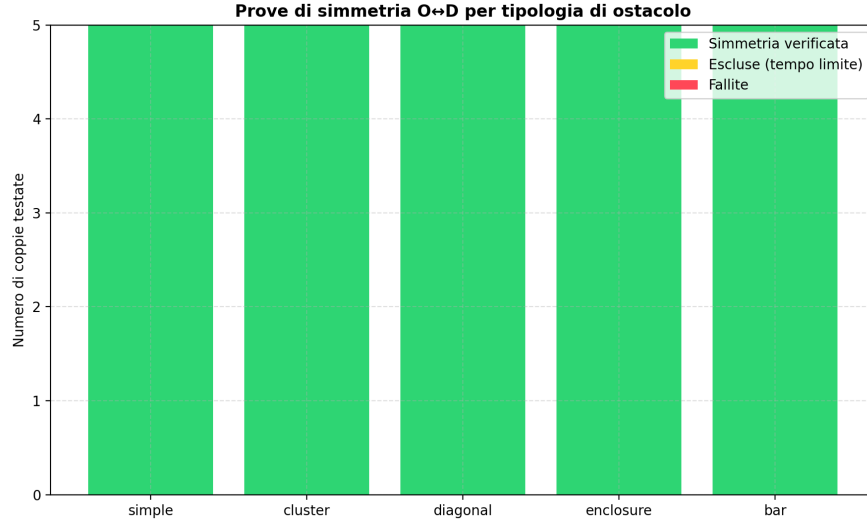


Figura 14: Esito delle prove di simmetria $O \leftrightarrow D$ per ciascuna tipologia di ostacolo.

5.5 Oracolo di correttezza indipendente: confronto con A^*

La verifica di simmetria appena discussa è una condizione necessaria ma non sufficiente di correttezza: un'implementazione potrebbe restituire la stessa lunghezza sbagliata in entrambe le direzioni. Per un controllo più forte, seguendo il suggerimento della specifica di confrontare implementazioni diverse dello stesso calcolo (diapositiva 63 del testo dell'elaborato), è stata realizzata una seconda implementazione indipendente della ricerca del cammino minimo: l'algoritmo A^* [4] applicato direttamente al grafo delle celle (`src/astar.py`), con costo 1 sui passi cardinali e $\sqrt{2}$ su quelli diagonali e con la stessa semantica di attraversamento dei cammini liberi (il passo diagonale fra due celle libere è sempre ammesso, diapositiva 3).

La chiave dell'oracolo è che la distanza ideale d_{lib} già definita nel Compito 2 è esattamente la distanza octile, che su questo grafo è un'euristica ammissibile e consistente: A^* restituisce quindi sempre la lunghezza ottima esatta, e lo fa in tempi dell'ordine dei millisecondi anche sulla griglia 200×200 , perché il suo costo è quasi lineare nel numero di celle e non risente del muro esponenziale della ricerca a landmark. A^* non sostituisce CAMMINOMIN, che la specifica impone nella forma pseu-

docodificata, ma ne convalida i risultati: ogni lunghezza completata da CAMMINOMIN deve coincidere con quella di A^* .

La verifica è duplice. In primo luogo la suite di test (`tests/test_astar.py`) confronta le due implementazioni su venti griglie casuali 15×15 di tipologie miste, inclusi i casi irraggiungibili, richiedendo l'uguaglianza entro 10^{-9} . In secondo luogo lo script `scripts/verifica_oracolo_astar.py` rigenera le stesse griglie della campagna di scaling (stessi semi e parametri) e confronta l'ottimo di A^* con le lunghezze già registrate in `scaling_results.json`, senza rieseguire la campagna: tutte le esecuzioni completate (le taglie 10 e 20 in ogni configurazione, il lato 50 a potatura forte in entrambe le direzioni, il lato 100 nella direzione di ritorno) coincidono con l'ottimo esatto. L'oracolo convalida così l'intera catena di calcolo esistente, dalla proiezione di raggi alla ricorsione.

5.6 Divario di ottimalità dei risultati interrotti (anytime)

L'oracolo consente anche di rispondere a una domanda che la campagna di scaling aveva lasciato aperta: quando la ricerca viene interrotta al tempo limite e restituisce il miglior cammino trovato fino a quel momento, *quanto* quel cammino dista dal vero ottimo? Senza un riferimento esatto il dato anytime è privo di scala; con A^* il divario diventa misurabile. La figura 15 riporta il divario percentuale per tutte le esecuzioni interrotte della campagna (i dati numerici sono in `divario_anytime.json`).

Il quadro che emerge sfuma il significato del muro esponenziale. Al lato 100 la potatura forte, pur non riuscendo a completare la ricerca in 600 s, restituisce un cammino a solo lo 0,4% dall'ottimo (la debole al 4,3%); al lato 50 la debole interrotta è al 3,5%. Al lato 150 il divario cresce (dal 9,1% della forte al 19,6% della debole) e al lato 200 le configurazioni di andata non trovano alcun cammino completo, mentre la direzione di ritorno si ferma al 7,8% dall'ottimo. In altre parole: l'algoritmo interrotto è spesso un buon risolutore approssimato alle taglie medie, ma la qualità della soluzione anytime degrada, fino ad azzerarsi, alle taglie in cui la frontiera esplode. Anche questo risultato conferma l'utilità dell'ordinamento euristico della frontiera: i primi cammini trovati sono geometricamente ben orientati, ed è per

questo che il risultato parziale resta vicino all’ottimo finché almeno una foglia viene raggiunta.

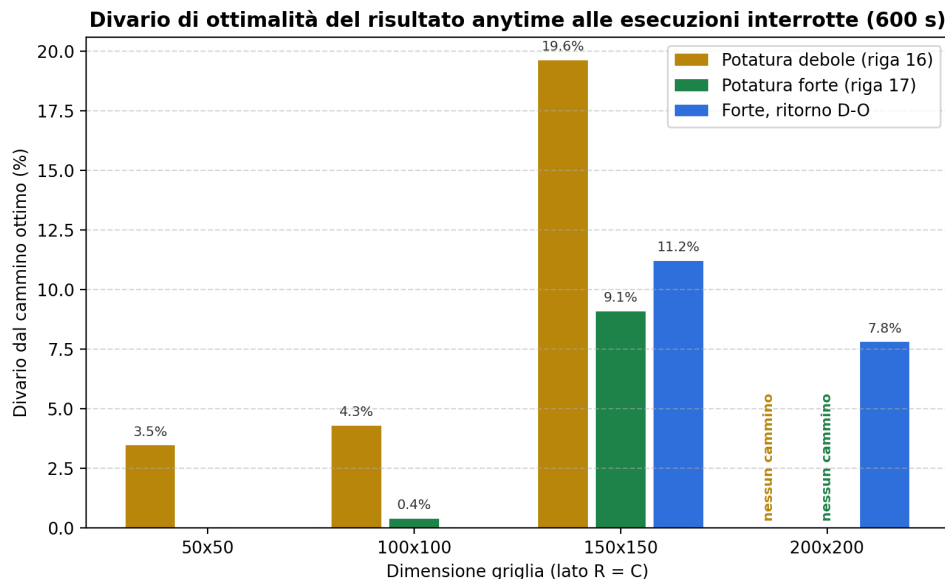


Figura 15: Divario percentuale dall’ottimo esatto (calcolato con A^*) dei cammini restituiti alle esecuzioni interrotte al tempo limite di 600 s.

5.7 Confronto di strutture dati: matrice di byte contro piani di bit

La specifica premia esplicitamente il confronto fra implementazioni diverse anche sul piano delle strutture dati e l’adozione di strutture che estendano le taglie elaborabili (diapositive 63 e 72 del testo dell’elaborato). La rappresentazione di riferimento della griglia è una matrice NumPy `uint8`, un byte per cella; gli stati logici necessari sono però soltanto tre (cella libera, ostacolo fisso, ostacolo temporaneo), che si codificano in due piani di bit paralleli, uno per gli ostacoli fissi e uno per le marcature temporanee: due bit per cella, un quarto della memoria. La classe `BitPackedGrid` (`src/bitgrid.py`) impacchetta i due piani in array di byte (otto celle per byte) ed espone la stessa interfaccia di accesso della matrice: tutti gli algoritmi esistenti, dal

calcolo delle chiusure a CAMMINOMIN, operano sull'oggetto impacchettato senza alcuna modifica, e i test di equivalenza (`tests/test_bitgrid.py`) confermano chiusure, frontiere e risultati identici.

L'esperimento dedicato (`scripts/verifica_griglia_bit.py`) quantifica il compromesso su taglie crescenti. La memoria si riduce del fattore atteso (a 200×200 : 40 000 byte contro 10 000; a 400×400 : 160 000 contro 40 000), il che consente, a parità di memoria disponibile, griglie di lato doppio. Il prezzo è il tempo: l'accesso alla singola cella richiede operazioni di scorporo dei bit in Python puro anziché un'indicizzazione diretta, e sia il calcolo di una chiusura sia un'esecuzione completa di CAMMINOMIN risultano circa due volte più lenti. La valutazione critica è quindi netta: per le taglie trattate in questa campagna la matrice di byte resta la scelta corretta, perché la memoria non è mai il collo di bottiglia (poche decine di kilobyte anche a lato 200) mentre il tempo lo è sempre; la rappresentazione a bit diventa preferibile solo in scenari con vincoli di memoria stringenti o griglie di lato molto maggiore, dove il fattore quattro di risparmio sposta il limite delle taglie rappresentabili.

6 Conclusioni

In questa relazione abbiamo documentato l’implementazione e la sperimentazione di un sistema per la ricerca del cammino a otto direzioni su griglie bidimensionali. Il software è stato sviluppato in Python utilizzando la libreria NumPy per la gestione vettoriale della griglia.

6.1 Limitazioni riscontrate

L’implementazione presenta le seguenti limitazioni, emerse dalle prove di esecuzione:

1. **Scalabilità della ricerca esaustiva:** su griglie con densità di ostacoli del 20%, le interrogazioni angolo-angolo si completano in pochi secondi fino al lato 50×50 (circa 1,6 s), ma da 100×100 in su la ricerca esaustiva del cammino ottimo supera il tempo limite in tutte le configurazioni provate, richiedendo decine di milioni di chiamate ricorsive. Per accertare se si trattasse di un limite strutturale o di un semplice budget insufficiente, il tempo limite è stato decuplicato a 600 s (10 minuti) senza alcun beneficio: da lato 100 in su nessuna configurazione completa la ricerca nemmeno con dieci volte più tempo (si veda il Compito 4), e al lato 200 né la potatura debole né quella forte trovano un cammino completo qualsiasi entro il budget concesso. In questi casi l’algoritmo restituisce comunque il miglior cammino individuato (quando esiste), senza però dimostrarne l’ottimalità entro il tempo concesso. Grazie all’oracolo A* introdotto nel Compito 4 questa distanza dall’ottimo è ora quantificata: i cammini anytime restano entro pochi punti percentuali dall’ottimo alle taglie medie (dallo 0,4% al 4,3% fino al lato 100), ma degradano alle taglie maggiori (fino al 19,6% al lato 150, nessun cammino trovato in andata al lato 200). Un caso patologico è stato invece eliminato del tutto: le interrogazioni su coppie irraggiungibili, che in precedenza esaurivano il tempo limite senza risposta, vengono ora risolte in tempo $O(R \cdot C)$ dal controllo preliminare dei componenti connessi (si veda il Compito 3). Le ottimizzazioni adottate (potatura forte e ordinamento euristico della frontiera) ne riducono le costanti anche di diver-

si ordini di grandezza, ma non l'andamento asintotico: poiché i risultati dei sotto-problemi non vengono memorizzati (la memorizzazione è stata rimossa per il difetto logico descritto nel Compito 3), sulle griglie molto ostruite le stesse chiusure intermedie vengono ricalcolate più volte;

2. **Limite di profondità della pila di ricorsione:** l'architettura ricorsiva fa affidamento sulla pila delle chiamate gestita dall'interprete. Sebbene in Python sia possibile innalzare questo limite tramite la chiamata `sys.setrecursionlimit(15000)` [3], per percorsi con migliaia di landmark intermedi, il sistema rischia di incorrere in un errore di `RecursionError`. Per verificare empiricamente questo rischio abbiamo costruito una griglia "a pettine" (`scripts/verifica_limite_ricorsione.py`): barriere verticali a piena altezza con un unico varco che alterna fra il bordo superiore e quello inferiore, forzando un cammino a zig-zag con un landmark per barriera. Su questa istanza la profondità di ricorsione cresce linearmente col numero di barriere, ma il numero di chiamate ricorsive cresce in modo esponenziale (da 4 640 chiamate con 12 barriere a 271 163 con 18, un fattore prossimo a 2 per barriera aggiuntiva): con un budget di 30 s la ricerca raggiunge appena una profondità massima di 55 prima di esaurire il tempo, ben lontana dal limite di 15 000. Nella pratica, quindi, il costo computazionale della ricerca (si veda la limitazione precedente) è il vincolo che si manifesta per primo: non siamo riusciti a costruire un'istanza che raggiunga profondità di migliaia di landmark entro un tempo ragionevole, il che rende il rischio di `RecursionError` perlopiù teorico per le dimensioni di griglia effettivamente praticabili.

6.2 Motivazioni delle scelte implementative

Le scelte di progettazione adottate mirano a contenere l'occupazione di memoria e a limitare il numero di nodi esplorati:

- **Gestione della memoria:** la rappresentazione della griglia come matrice NumPy di tipo `uint8` alloca un solo byte contiguo per cella. Unita al ritracciamento sul posto, che riusa l'unica matrice condivisa invece di duplicarla a

ogni livello, questa scelta evita la proliferazione di oggetti tipica di un modello in cui ogni cella è un'istanza distinta. La scelta è stata inoltre convalidata dal confronto sperimentale con la rappresentazione alternativa a piani di bit (Compito 4): quest'ultima riduce la memoria di quattro volte ma rallenta il calcolo di circa due volte, e poiché alle taglie praticabili la memoria non è mai il collo di bottiglia, la matrice di byte resta la rappresentazione predefinita;

- **Contenimento del tempo di calcolo:** la potatura branch-and-bound globale e l'ordinamento euristico della frontiera mirano a sfoltire l'albero di esplorazione. Sulle istanze risolubili entro il tempo limite questi accorgimenti riducono le costanti di calcolo; sulle griglie più grandi, però, non bastano a evitare il superamento del tempo limite (si veda il Compito 4).

6.3 Sviluppi futuri

Possibili sviluppi futuri includono:

- **Conversione in un algoritmo iterativo con pila esplicita:** la trasformazione della ricorsione in una struttura iterativa basata su una pila gestita esplicitamente in memoria dinamica eliminerebbe il legame con la pila delle chiamate dell'interprete. Questo consentirebbe di elaborare cammini di qualsiasi lunghezza o profondità senza il rischio di esaurire la pila.

Si potrebbe inoltre riconsiderare la reintroduzione di una memoria dei sotto-problemi, questa volta risolvendo il difetto logico che ne aveva imposto la rimozione, ossia la dipendenza del risultato dalla lunghezza accumulata delle chiamate esterne, non inclusa nella chiave. Per limitarne l'occupazione, tale memoria andrebbe dotata di capacità massima e di una politica di sfratto come *Least Recently Used* (LRU) o *Least Frequently Used* (LFU), in modo da rimuovere automaticamente gli stati meno utilizzati una volta raggiunta la soglia.

7 Uso del codice

Questo capitolo fornisce il manuale utente completo e la documentazione tecnica per configurare, eseguire e validare il software sviluppato. Il punto di ingresso principale è l'interfaccia a riga di comando scritta in Python, che implementa tutti i compiti dell'elaborato.

7.1 Configurazione e requisiti di sistema

Il software è scritto in Python 3.13. Per eseguire l'applicazione e riprodurre i risultati sperimentali è necessario configurare l'ambiente virtuale installando le dipendenze dichiarate:

```
# Creazione dell'ambiente virtuale
python3 -m venv .venv

# Attivazione dell'ambiente
source .venv/bin/activate

# Installazione delle librerie necessarie (NumPy, Matplotlib)
pip install -r requirements.txt
```

7.2 Configurazione del file JSON delle griglie

Il file JSON utilizzato per memorizzare e caricare la struttura delle griglie adotta una sintassi strutturata, che dichiara esplicitamente le dimensioni della matrice, le tipologie di ostacoli con cui la griglia è stata generata (riportate poi nel riassunto dell'esecuzione, come richiesto dalla diapositiva 71 del testo dell'elaborato) e le coordinate bidimensionali di ciascuna cella ostacolo fisso:

```
{
  "rows": 50,
  "cols": 50,
```

```

"types": ["simple", "cluster"],
"obstacles": [
    [0, 5],
    [0, 6],
    [1, 10],
    ...
]
}

```

7.3 Guida all'uso della riga di comando

Tutte le funzionalità del software sono esposte tramite il modulo principale `main.py`. I comandi devono essere eseguiti con l'interprete dell'ambiente virtuale e includendo la variabile d'ambiente `PYTHONPATH=.`

7.3.1 1. Generazione di griglie (Compito 1)

Per generare proceduralmente ostacoli misti (`simple`, `cluster`, `diagonal`, `enclosure`, `bar` o lo scenario combinato `mix`) con densità configurabile e salvare l'esito in formato JSON e in immagine PNG ad alta risoluzione:

```

PYTHONPATH=. .venv/bin/python main.py generate --rows 50 --cols 50 \
--types mix --density 0.2 --seed 42 \
-o grids/grid.json --save-img results/test_grid_generated.png

```

7.3.2 2. Calcolo di contesto, complemento e frontiera (Compito 2)

Per tracciare i raggi geometrici dall'origine e calcolare l'insieme delle celle appartenenti al contesto (tipo 1), al complemento (tipo 2) e alla frontiera locale:

```

PYTHONPATH=. .venv/bin/python main.py context --grid grids/grid.json \
--origin 10,10 --type both

```

7.3.3 3. Calcolo della distanza ideale d_{lib}

Per calcolare la distanza minima teorica priva di ostacoli su una griglia con adiacenza a otto direzioni tramite la formula geometrica chiusa:

```
PYTHONPATH=. .venv/bin/python main.py dlib --origin 10,10 --dest 40,40
```

7.3.4 4. Calcolo del cammino minimo (Compito 3)

Per calcolare il cammino minimo tramite ritracciamento ricorsivo basato su landmark (con la potatura branch-and-bound globale attiva), esportare l'immagine PNG ad alta risoluzione del percorso e stampare il riassunto JSON strutturato (diapositiva 71):

```
PYTHONPATH=. .venv/bin/python main.py camminomin --grid grids/grid.json \
--origin 0,0 --dest 49,49 --strong-pruning --summary \
--save-img results/test_path_resolved.png
```

L'inclusione del parametro `-strong-pruning` attiva la potatura tramite riga 17, mentre la sua assenza mantiene attiva la riga 16. Il controllo preliminare dei componenti connessi per le coppie irraggiungibili (si veda il Compito 3) è attivo in modo predefinito e si disattiva con `-no-component-check`, utile per riprodurre il confronto sperimentale. Il parametro `-summary` stampa il resoconto strutturato con il conteggio delle celle di frontiera analizzate, il numero di potature eseguite e l'altezza massima raggiunta dalla pila ricorsiva. Il campo `memoria_rappresentazione_byte` del resoconto riporta la dimensione statica della matrice (righe $\times$ colonne), non il picco di memoria effettivamente utilizzato durante quella singola ricerca: quest'ultimo è misurato con `tracemalloc` solo nella campagna sperimentale del Compito 4, per non raddoppiare i tempi di ogni singola invocazione (si veda la nota metodologica del Compito 4).

7.3.5 5. Campagna sperimentale completa (Compito 4)

Per lanciare l'intera batteria automatizzata di prove sperimentali su griglie scalabili (fino a 200×200), densità variabili e strategie di potatura, esportando i sei grafici analitici discussi nel capitolo precedente nella cartella dei risultati:

```
PYTHONPATH=. .venv/bin/python main.py experiment --output-dir results
```

7.3.6 6. Verifica di simmetria formale

Per lanciare la verifica automatica di simmetria ($O \leftrightarrow D$ contro $D \leftrightarrow O$) su un numero N di coppie casuali di nodi:

```
PYTHONPATH=. .venv/bin/python main.py experiment --verify-symmetry-count 10
--symmetry-grid-size 20
```

7.4 Rigenerazione delle figure pedagogiche della relazione

Le figure dimostrative della relazione (panoramica del sistema, tipologie di ostacolo, cammini liberi di tipo 1/2, contesto/complemento/frontiera con i raggi, mappa di esplorazione della potatura) non provengono dalla campagna sperimentale del Compito 4, ma da cinque script indipendenti nella cartella `scripts/`, ciascuno eseguibile singolarmente e che riusa direttamente i moduli di `src/` senza introdurre logica nuova:

```
PYTHONPATH=. .venv/bin/python scripts/genera_immagine_panoramica.py
PYTHONPATH=. .venv/bin/python scripts/genera_immagine_tipologie_ostacoli.py
PYTHONPATH=. .venv/bin/python scripts/genera_immagine_cammini_liberi.py
PYTHONPATH=. .venv/bin/python scripts/genera_immagine_contesto.py
PYTHONPATH=. .venv/bin/python scripts/genera_immagine_esplorazione_potatura.py
```

Ciascuno scrive il proprio PNG in `Relazione/images/`, sovrascrivendo il file esistente. Altri due script non producono immagini, ma riproducono verifiche numeriche citate in prosa nel Compito 4 e nelle Conclusioni: la verifica empirica del limite di ricorsione e il confronto supplementare fra potatura debole e forte su una griglia 100×100 :

```
PYTHONPATH=. .venv/bin/python scripts/verifica_limite_ricorsione.py
PYTHONPATH=. .venv/bin/python scripts/verifica_potatura_100.py
```

Tre ulteriori script riproducono gli esperimenti delle estensioni discusse nei Capitoli 3 e 4: l'effetto del controllo dei componenti connessi su una coppia irraggiungibile (attenzione: la configurazione senza controllo esaurisce l'intero tempo limite di 600 s), la verifica dell'oracolo A* con il calcolo del divario anytime (che genera anche la figura `divario_anytime.png` e il file `divario_anytime.json`) e il confronto di memoria e tempo fra la matrice di byte e la griglia a piani di bit:

```
PYTHONPATH=. .venv/bin/python scripts/verifica_componenti.py
PYTHONPATH=. .venv/bin/python scripts/verifica_oracolo_astar.py
PYTHONPATH=. .venv/bin/python scripts/verifica_griglia_bit.py
```

7.5 Esecuzione della batteria di prove unitarie

Il progetto include una copertura completa di prove unitarie automatiche, che validano l'esattezza di tutti i moduli geometrici e logici (distanza libera, proiezione di raggi, compattazione, ritracciamento sul posto, generazione procedurale degli ostacoli, salvataggio e caricamento della griglia). Per avviare le prove unitarie:

```
PYTHONPATH=. .venv/bin/python -m unittest discover -s tests -p "test_*.py"
```

Riferimenti bibliografici

- [1] NumPy Developers, *Data type objects (dtype)*, documentazione ufficiale NumPy. <https://numpy.org/doc/stable/reference/arrays.dtypes.html>
- [2] Python Software Foundation, *Sorting HOW TO*, documentazione ufficiale Python (algoritmo Timsort, complessità $O(n \log n)$). <https://docs.python.org/3/howto/sorting.html>
- [3] Python Software Foundation, *sys.setrecursionlimit*, documentazione ufficiale Python. <https://docs.python.org/3/library/sys.html#sys.setrecursionlimit>
- [4] P. E. Hart, N. J. Nilsson, B. Raphael, *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*, IEEE Transactions on Systems Science and Cybernetics, vol. 4, n. 2, pp. 100–107, 1968.